

Experiment-02

Aim:

A. Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are: - “Low Salary”: All monthly salaries strictly less than \$20,000. - “Average Salary”: All monthly salaries in the inclusive range [\$20,000, \$50,000]. - “High Salary”: All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

B. Two legacy HR systems (A & B) have separate records of employee salaries. These records may overlap. Management wants to merge these datasets and identify each unique employee (by EmpID) along with their lowest recorded salary across both systems. Objective: - Combine two tables A and B - Return each EmpID with their lowest salary, and the corresponding Ename.

C. A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report. If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the Department Name, Employee Name, and Salary, arranged in ascending order of department name.

D. Find the 2nd highest earning employee from the same schema.

Tools / Platform Used:

- Windows
- PostgreSQL
- pgAdmin

Theory:

A subquery is a SQL query nested inside a larger statement, such as a SELECT, INSERT, UPDATE, or DELETE statement. Also known as an inner query or nested query, it acts as a helper to pass dynamically retrieved data to the primary outer query.

Types of Subqueries are: - Scalar Subquery – returns exactly one value for the output - Multi-row Subquery – returns multiple rows but only one column - Multi-column – returns multiple columns and rows - Correlated – subquery which is linked to the outer query

Code and Output:

A.

```
CREATE TABLE bank_accounts (  
    account_id INT PRIMARY KEY,  
    monthly_salary INT NOT NULL  
);
```

```
INSERT INTO bank_accounts (account_id, monthly_salary) VALUES  
(201, 18500),  
(202, 27000),  
(203, 68200),  
(204, 20000),  
(205, 50000),  
(206, 11400);
```

```
1 CREATE TABLE bank_accounts (  
2     account_id INT PRIMARY KEY,  
3     monthly_salary INT NOT NULL  
4 );  
5  
6 INSERT INTO bank_accounts (account_id, monthly_salary) VALUES  
7 (201, 18500),  
8 (202, 27000),  
9 (203, 68200),  
0 (204, 20000),  
1 (205, 50000),  
2 (206, 11400);  
3
```

STDIN
Input for the program (Optional)

Output:
CREATE TABLE
INSERT 0 6

```
SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count  
FROM bank_accounts WHERE monthly_salary < 20000
```

UNION ALL

```
SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count  
FROM bank_accounts WHERE monthly_salary >= 20000 AND monthly_salary <= 50000
```

UNION ALL

```
SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count  
FROM bank_accounts WHERE monthly_salary > 50000;
```

```

1 CREATE TABLE bank_accounts (
2     account_id INT PRIMARY KEY,
3     monthly_salary INT NOT NULL
4 );
5
6 INSERT INTO bank_accounts (account_id, monthly_salary) VALUES
7 (201, 18500),
8 (202, 27000),
9 (203, 68200),
0 (204, 20000),
1 (205, 50000),
2 (206, 11400);
3
4 SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count
5 FROM bank_accounts WHERE monthly_salary < 20000
6
7 UNION ALL
8
9 SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count
10 FROM bank_accounts WHERE monthly_salary >= 20000 AND monthly_salary <= 50000
11
12 UNION ALL
13
14 SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count
15 FROM bank_accounts WHERE monthly_salary > 50000;
16

```

STDIN
Input for the program (optional)

Output:

```

CREATE TABLE
INSERT 0 6

```

category	accounts_count
Low Salary	2
Average Salary	3
High Salary	1

(3 rows)

B.

```

CREATE TABLE hr_system_a (
    emp_id INT,
    ename VARCHAR(50),
    salary INT
);

```

```

CREATE TABLE hr_system_b (
    emp_id INT,
    ename VARCHAR(50),
    salary INT
);

```

```

INSERT INTO hr_system_a (emp_id, ename, salary) VALUES
(1, 'Rohan', 55000),
(2, 'Meera', 32000),
(3, 'Farah', 41000);

```

```

INSERT INTO hr_system_b (emp_id, ename, salary) VALUES
(2, 'Meera', 29000),
(3, 'Farah', 47000),
(4, 'Ishaan', 38000);

```

```

1 CREATE TABLE hr_system_a (
2     emp_id INT,
3     ename VARCHAR(50),
4     salary INT
5 );
6
7 CREATE TABLE hr_system_b (
8     emp_id INT,
9     ename VARCHAR(50),
10    salary INT
11 );
12
13 INSERT INTO hr_system_a (emp_id, ename, salary) VALUES
14 (1, 'Rohan', 55000),
15 (2, 'Meera', 32000),
16 (3, 'Farah', 41000);
17
18 INSERT INTO hr_system_b (emp_id, ename, salary) VALUES
19 (2, 'Meera', 29000),
20 (3, 'Farah', 47000),
21 (4, 'Ishaan', 38000);
22

```

STDIN

Input for the program (Optional)

Output:

```

CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 3

```

```

SELECT emp_id, MIN(ename) AS ename, MIN(salary) AS lowest_salary
FROM (
    SELECT * FROM hr_system_a
    UNION ALL
    SELECT * FROM hr_system_b
) AS merged
GROUP BY emp_id
ORDER BY emp_id;

```

```

    salary INT
);

CREATE TABLE hr_system_b (
    emp_id INT,
    ename VARCHAR(50),
    salary INT
);

INSERT INTO hr_system_a (emp_id, ename, salary) VALUES
(1, 'Rohan', 55000),
(2, 'Meera', 32000),
(3, 'Farah', 41000);

INSERT INTO hr_system_b (emp_id, ename, salary) VALUES
(2, 'Meera', 29000),
(3, 'Farah', 47000),
(4, 'Ishaan', 38000);
SELECT emp_id, MIN(ename) AS ename, MIN(salary) AS lowest_salary
FROM (
    SELECT * FROM hr_system_a
    UNION ALL
    SELECT * FROM hr_system_b
) AS merged
GROUP BY emp_id
ORDER BY emp_id;

```

Output:

```

CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 3

```

emp_id	ename	lowest_salary
1	Rohan	55000
2	Meera	29000
3	Farah	41000
4	Ishaan	38000

(4 rows)

C.

```

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

```

```

CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary INT,
    dept_id INT REFERENCES department(dept_id)
);

```

```
);
```

```
INSERT INTO department VALUES
```

```
(1, 'IT'),  
(2, 'HR');
```

```
INSERT INTO employee VALUES
```

```
(1, 'Nikhil', 72000, 1),  
(2, 'Diya', 85000, 1),  
(3, 'Vikram', 85000, 1),  
(4, 'Tara', 60000, 2),  
(5, 'Arjun', 65000, 2);
```

```
CREATE TABLE department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary INT,  
    dept_id INT REFERENCES department(dept_id)  
);
```

```
INSERT INTO department VALUES  
(1, 'IT'),  
(2, 'HR');
```

```
INSERT INTO employee VALUES  
(1, 'Nikhil', 72000, 1),  
(2, 'Diya', 85000, 1),  
(3, 'Vikram', 85000, 1),  
(4, 'Tara', 60000, 2),  
(5, 'Arjun', 65000, 2);
```

```
SELECT d.dept_name, e.emp_name, e.salary  
FROM employee AS e  
JOIN department AS d ON e.dept_id = d.dept_id  
WHERE e.salary = (  
    SELECT MAX(salary) FROM employee WHERE dept_id = e.dept_id  
)  
ORDER BY d.dept_name;
```

STDIN

Input for the program (Optional)

Output:

```
CREATE TABLE  
CREATE TABLE  
INSERT 0 2  
INSERT 0 5
```

```

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary INT,
    dept_id INT REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR');

INSERT INTO employee VALUES
(1, 'Nikhil', 72000, 1),
(2, 'Diya', 85000, 1),
(3, 'Vikram', 85000, 1),
(4, 'Tara', 60000, 2),
(5, 'Arjun', 65000, 2);
SELECT d.dept_name, e.emp_name, e.salary
FROM employee AS e
JOIN department AS d ON e.dept_id = d.dept_id
WHERE e.salary = (
    SELECT MAX(salary) FROM employee WHERE dept_id = e.dept_id
)
ORDER BY d.dept_name;

```

STDIN

Input for the program (Optional)

Output:

```

CREATE TABLE
CREATE TABLE
INSERT 0 2
INSERT 0 5
dept_name | emp_name | salary
-----+-----+-----
HR         | Arjun    | 65000
IT         | Diya     | 85000
IT         | Vikram   | 85000
(3 rows)

```

D.

```

SELECT *
FROM employee
WHERE salary = (
    SELECT MAX(salary)
    FROM employee
    WHERE salary < (SELECT MAX(salary) FROM employee)
);

```

```

CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary INT,
    dept_id INT REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR');

INSERT INTO employee VALUES
(1, 'Nikhil', 72000, 1),
(2, 'Diya', 85000, 1),
(3, 'Vikram', 85000, 1),
(4, 'Tara', 60000, 2),
(5, 'Arjun', 65000, 2);
SELECT *
FROM employee
WHERE salary = (
    SELECT MAX(salary)
    FROM employee
    WHERE salary < (SELECT MAX(salary) FROM employee)
);

```

Output:

```

CREATE TABLE
CREATE TABLE
INSERT 0 2
INSERT 0 5
emp_id | emp_name | salary | dept_id
-----+-----+-----+-----
1      | Nikhil   | 72000  | 1
(1 row)

```

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with subqueries in PostgreSQL.

- Understood the concept of subqueries.